Quick Sort

```
function Quicksort (L[low:high])
   if high>low then
      call Rearrange (L[low:high], position)
      call Quicksort (L[low:position-1])
      call Quicksort (L[position+1:high])
   endif
end
```

```
function Rearrange (L[low:high],position)
   right ← low
   left ← high+1
   x ← L[low]
   while right<left do
      repeat right ← right+1 until L[right]≥x
      repeat left ← left-1 until L[left]≤x
      if right<left then
         call Interchange (L[right], L[left])
      endif
   endwhile
   position ← left
   L[low] ← L[position]
   L[position] ← x
end
```

Initial call:
call Quicksort (L[0:n-1])

Puts pivot $L[low]$ into proper position $L[position]$ and returns *position*

Elements on the left of $L[position] \leq L[position]$

Elements on the right of $L[position] \geq L[position]$

Sort the two sublists

Pivot:
first element in the list

Quick Sort

```
function Quicksort (L[low:high])
   if high>low then
      call Rearrange (L[low:high], position)
      call Quicksort (L[low:position-1])
      call Quicksort (L[position+1:high])
   endif
end
```

```
function Rearrange (L[low:high],position)
   right ← low
   left ← high+1
   x ← L[low]
   while right<left do
      repeat right ← right+1 until L[right]≥x
      repeat left ← left-1 until L[left]≤x
      if right<left then
         call Interchange (L[right], L[left])
      endif
   endwhile
   position ← left
   L[low] ← L[position]
   L[position] ← x
end
```

Pivot:
first element in the list

Other choices?

- Choose randomly
- Median of three – leftmost, rightmost, middle

What is the use?

Quick Sort

```
function Quicksort (L[low:high])
   if high>low then
      call Rearrange (L[low:high], position)
      call Quicksort (L[low:position-1])
      call Quicksort (L[position+1:high])
   endif
end
```

```
function Rearrange (L[low:high],position)
   right ← low
   left ← high+1
   x ← L[low]
   while right<left do
      repeat right ← right+1 until L[right]≥x
      repeat left ← left-1 until L[left]≤x
      if right<left then
         call Interchange (L[right], L[left])
      endif
   endwhile
   position ← left
   L[low] ← L[position]
   L[position] ← x
end
```

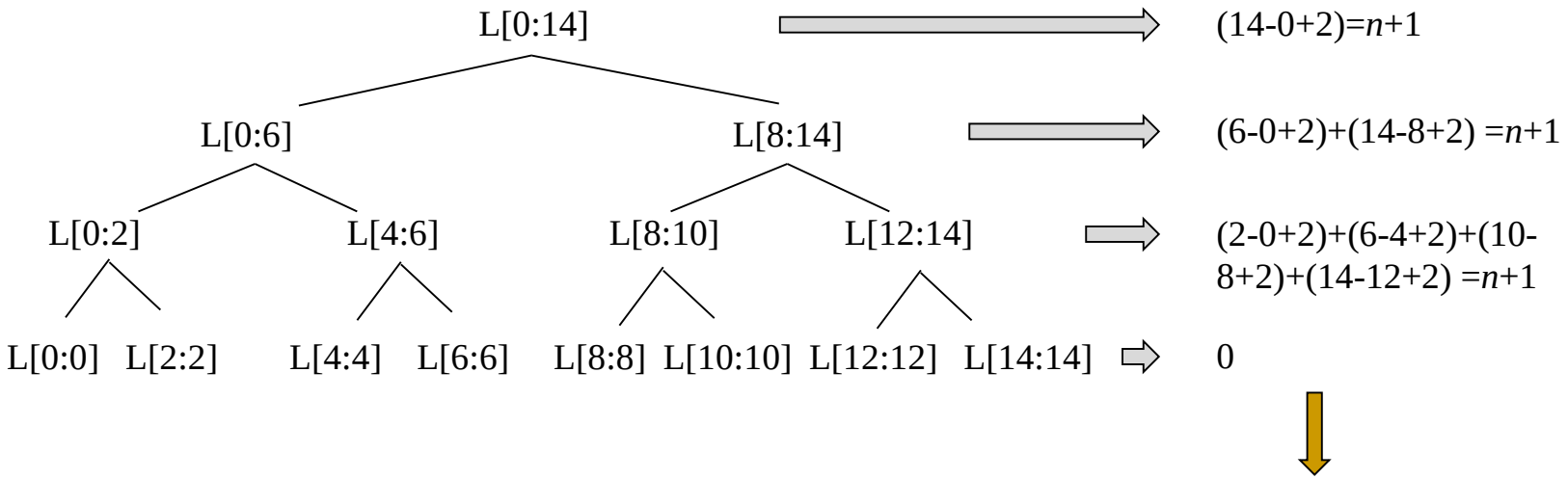
What is the basic operation?

exactly
$high - low + 2$
basic operations

Quick Sort

- $B(n)$
  - List is divided into two sublists of equal size
    Assume $n = 2^k - 1$
  - Example: L[0:14]

What is best input in divide and conquer algorithms?
And why?



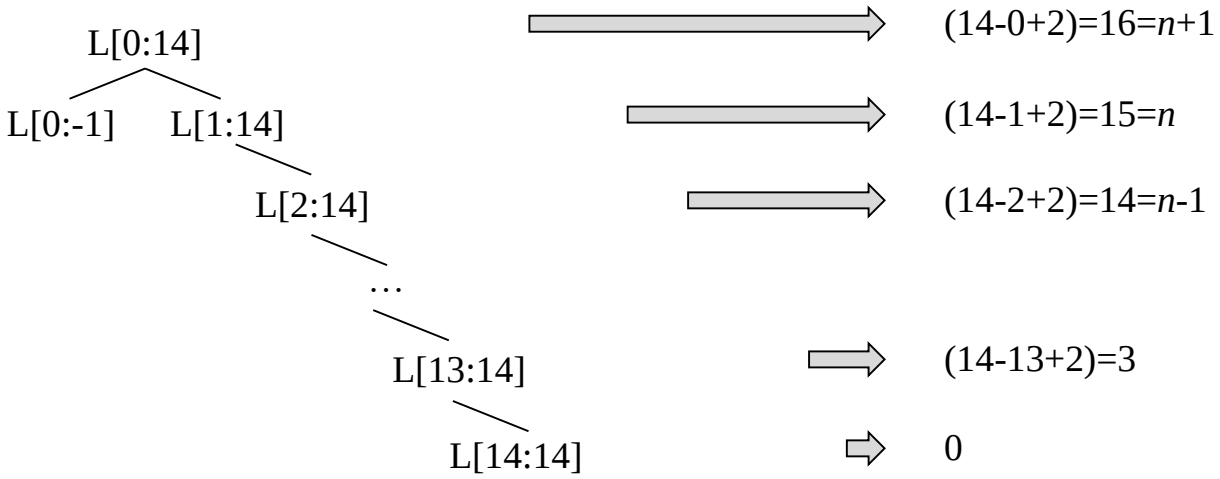
$$B(n) = (k-1)(n+1) = (\log_2(n+1) - 1)(n+1)$$
$$\in \theta(n \log n) \ , n = 2^k - 1$$
Interpolation $\rightarrow B(n) \in \theta(n \log n)$, $n \in \mathbb{N}$

k levels
k-1 $\Rightarrow$ *n*+1 comparisons

Quick Sort

- $W(n)$
  - List is divided into two sublists of maximally unequal size
  - Already sorted list
  - Example: L[0:14]

What is worst input in divide and conquer algorithms?
And why?



$W(n) = \sum_{i=3}^{n+1} i \in \theta(n^2)$

Quick Sort

- $A(n)$
  - Assume that it is equally likely that pivot ($L[low]$) will be placed in any position after Rearrange
  - View the algorithm as having two steps

```
function Quicksort (L[low:high])
  if high>low then
    call Rearrange (L[low:high], position)
    call Quicksort (L[low:position-1])
    call Quicksort (L[position+1:high])
  endif
end
```

```
function Rearrange (L[low:high],position)
  right ← low
  left ← high+1
  x ← L[low]
  while right<left do
    repeat right ← right+1 until L[right]≥x
    repeat left ← left-1 until L[left]≤x
    if right<left then
      call Interchange (L[right], L[left])
    endif
  endwhile
  position ← left
  L[low] ← L[position]
  L[position] ← x
end
```

τ_1 : number of operations in Rearrange

τ_2 : number of operations in recursive calls

$$\Rightarrow \tau = \tau_1 + \tau_2$$

$$A(n) = E[\tau] = E[\tau_1] + E[\tau_2]$$
$$= (n+1) + E[\tau_2]$$

Simplify:
Divide into steps

Quick Sort

- $A(n)$
  - $A(n) = (n+1) + E[\tau_2]$



After pivot is placed into proper position,
average number of operations in recursive calls

Conditional expectation

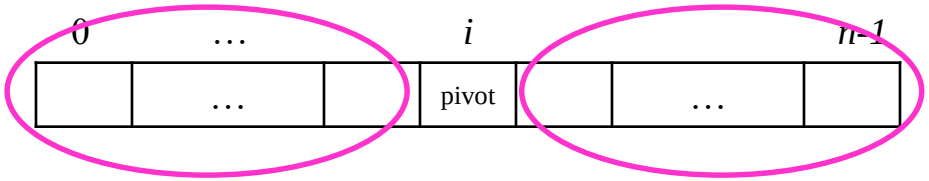
$E[\tau_2] = \sum_x E[\tau_2 | X = x] P(X = x)$ X : position of pivot

$E[\tau_2] = \sum_{i=0}^{n-1} E[\tau_2 | X = i] P(X = i)$

Simplify:
Condition on some info

Quick Sort

- $A(n)$
  - $A(n) = (n+1) + \sum_{i=0}^{n-1} E[\tau_2 | X = i] P(X = i)$



call Quicksort (L[0:i-1]) call Quicksort (L[i+1:n-1])

$E[\tau_2 | X = i] \rightarrow ???$ + $E[\tau_2 | X = i] \rightarrow ???$

$E[\tau_2 | X = i] \rightarrow A(i)$ + $E[\tau_2 | X = i] \rightarrow A(n-i-1)$

Quick Sort

- $A(n)$
  - $A(n) = (n+1) + \sum_{i=0}^{n-1} E[\tau_2 | X = i] P(X = i)$

$P(X = i) \rightarrow {}^{1}/_{n}$

$$= (n+1) + \sum_{i=0}^{n-1} [A(i) + A(n-i-1)] * \frac{1}{n}$$

$$= (n+1) + \frac{2}{n}[A(0) + A(1) + \cdots + A(n-1)] \quad , \; A(0) = A(1) = 0$$

full history recurrence relation

Quick Sort

- $A(n)$

  - $A(n) = (n+1) + \frac{2}{n}[A(0) + A(1) + \cdots + A(n-1)] \quad , \; A(0) = A(1) = 0$



$$nA(n) = n(n+1) + 2[A(0) + A(1) + \cdots + A(n-1)]$$
$$-\quad (n-1)A(n-1) = (n-1)n + 2[A(0) + A(1) + \cdots + A(n-2)]$$

$$nA(n) - (n-1)A(n-1) = 2n + 2A(n-1)$$

$$\frac{A(n)}{n+1} = \frac{A(n-1)}{n} + \frac{2}{n+1}$$

$\frac{A(n)}{n+1}$ → $t(n)$

$$\Rightarrow t(n) = t(n-1) + \frac{2}{n+1} \; , \; t(1) = 0$$
$$\Rightarrow t(n) = \sum_{i=2}^{n} \frac{2}{i+1} = 2H(n+1) - 3$$
$$\Rightarrow A(n) = t(n)(n+1) = (n+1)[2H(n+1) - 3] \in \theta(n \log n)$$

Quick Sort

- $B(n)$ & $A(n)$

$$B(n) = (n+1)(\log_2(n+1) - 1) = (n+1)\left[\frac{\log_e n+1}{\log_e 2} - 1\right] \cong (n+1)[1.44\ln(n+1) - 1]$$

$$A(n) \cong (n+1)[2\ln(n+1) - 3]$$

$$\Rightarrow \frac{A(n)}{B(n)} \cong 1.38$$

$A(n)$ & $B(n)$ are very close to each other
$A(n)$ just 38% more than $B(n)$

$A(n)$ & $W(n)$ are very different

Finding Min & Max

```
function MaxMin (L[0:n-1], Max, Min)
   Max ← L[0]
   Min ← L[0]
   for i←1 to n-1 do
      if L[i]>Max then
         Max ← L[i]
      else
         if L[i]<Min then
            Min ← L[i]
         endif
      endif
   endfor
end
```

Finding Min & Max

- $A(n)$
  - Assume input is a permutation of the numbers $(1,2,\ldots,n)$

    Example: n=5 $\Rightarrow$ Input: (4 1 5 2 3)

    No loss of generality
  - Assume each permutation is equally likely to occur
  - View the algorithm as having two steps

```
function MaxMin (L[0:n-1], Max, Min)
   Max ← L[0]
   Min ← L[0]
   for i←1 to n-1 do
      if L[i]>Max then
         Max ← L[i]
      else
         if L[i]<Min then
            Min ← L[i]
         endif
      endif
   endfor
end
```

τ_1 : number of times 1st comparison is executed

τ_2 : number of times 2nd comparison is executed

$\Rightarrow \tau = \tau_1 + \tau_2$

Simplify:
Divide into steps

Finding Min & Max

- $A(n)$
  - $A(n) = E[\tau] = E[\tau_1] + E[\tau_2]$

    $= (n-1) + E[n-1-X]$

    $= 2n - 2 - E[X]$

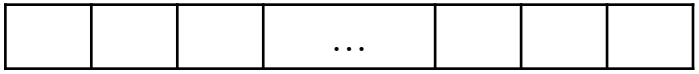
```
function MaxMin (L[0:n-1], Max, Min)
   Max ← L[0]
   Min ← L[0]
   for i←1 to n-1 do
      if L[i]>Max then
         Max ← L[i]
      else
         if L[i]<Min then
            Min ← L[i]
         endif
      endif
   endfor
end
```

Finding Min & Max

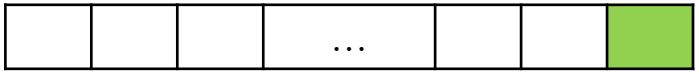
```
function MaxMin (L[0:n-1], Max, Min)
   Max ← L[0]
   Min ← L[0]
   for i←1 to n-1 do
      if L[i]>Max then
         Max ← L[i]
      else
         if L[i]<Min then
            Min ← L[i]
         endif
      endif
   endfor
end
```

- $A(n)$
  - $A(n) = 2n - 2 - E[X]$

Number of times 1st comparison is true



Difficult to compute ✗



If we know this value, easy ✓

  - Conditional expectation

$E[X]$ ✗ $E[X|Y = i]$ ✓

Simplify:
Condition on some info

Finding Min & Max

- $A(n)$
  - $A(n) = 2n - 2 - E[X]$



$$E[X] = \sum_{i=1}^{n} E[X|Y = i]P(Y = i)$$

$\frac{1}{n}$



$T(n)$



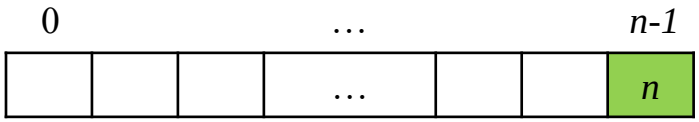
  - $A(n) = 2n - 2 - \sum_{i=1}^{n} E[X|Y = i]P(Y = i)$

```
function MaxMin (L[0:n-1], Max, Min)
  Max ← L[0]
  Min ← L[0]
  for i←1 to n-1 do
    if L[i]>Max then
      Max ← L[i]
    else
      if L[i]<Min then
        Min ← L[i]
      endif
    endif
  endfor
end
```

Finding Min & Max

```
function MaxMin (L[0:n-1], Max, Min)
   Max ← L[0]
   Min ← L[0]
   for i←1 to n-1 do
      if L[i]>Max then
         Max ← L[i]
      else
         if L[i]<Min then
            Min ← L[i]
         endif
      endif
   endfor
end
```

- $A(n)$
  - $E[X|Y=i]$
  - **Case 1**: $Y=n \Rightarrow E[X|Y=n]$



  - Example: n=4

$L[0:3]$

1	2	3	4
1	3	2	4
2	1	3	4
2	3	1	4
3	1	2	4
3	2	1	4

$\Rightarrow X=2$

$L[0:2]$

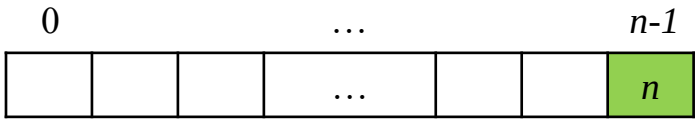
1	2	3
1	3	2
2	1	3
2	3	1
3	1	2
3	2	1

$\Rightarrow X=1$

Finding Min & Max

```
function MaxMin (L[0:n-1], Max, Min)
   Max ← L[0]
   Min ← L[0]
   for i←1 to n-1 do
      if L[i]>Max then
         Max ← L[i]
      else
         if L[i]<Min then
            Min ← L[i]
         endif
      endif
   endfor
end
```

- $A(n)$
  - $E[X|Y=i]$
  - **Case 1**: $Y=n \Rightarrow E[X|Y=n]$



  - Example: n=4

$L[0:3]$

1	2	3	4
1	3	2	4
2	1	3	4
2	3	1	4
3	1	2	4
3	2	1	4

$X=2$

2	3	1
3	1	2
3	2	1

$X=1$

$Y=n$

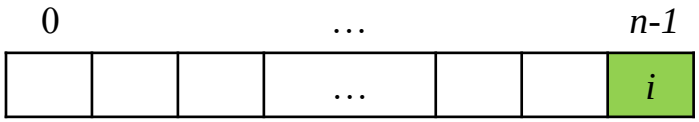
Number of times 1st comparison is true for a list of size n = Number of times 1st comparison is true for a list of size (n-1) + 1

$T(n)=T(n-1)+1$

Finding Min & Max

```
function MaxMin (L[0:n-1], Max, Min)
   Max ← L[0]
   Min ← L[0]
   for i←1 to n-1 do
      if L[i]>Max then
         Max ← L[i]
      else
         if L[i]<Min then
            Min ← L[i]
         endif
      endif
   endfor
end
```

- $A(n)$
  - $E[X|Y=i]$
  - **Case 2**: $Y=i \Rightarrow E[X|Y=i] \quad (1 \le i < n)$



  - Example: n=4 & Y=2

$L[0:3]$

1	2	4	2
1	4	3	2
3	1	4	2
3	4	1	2
4	1	3	2
4	3	1	2

$\Rightarrow X=1$

$L[0:2]$

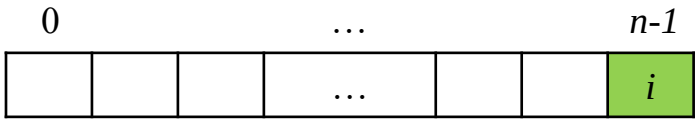
1	2	4
1	4	3
3	1	4
3	4	1
4	1	3
4	3	1

$\Rightarrow X=1$

Finding Min & Max

```
function MaxMin (L[0:n-1], Max, Min)
   Max ← L[0]
   Min ← L[0]
   for i←1 to n-1 do
      if L[i]>Max then
         Max ← L[i]
      else
         if L[i]<Min then
            Min ← L[i]
         endif
      endif
   endfor
end
```

- $A(n)$
  - $E[X|Y=i]$
  - **Case 2**: $Y=i \Rightarrow E[X|Y=i] \quad (1 \le i < n)$



  - Example: n=4 & Y=

$L[0:3]$

1	2	4	2
1	4	3	2
3	1	4	2
3	4	1	2
4	1	3	2
4	3	1	2

$X=1$

3	4	1
4	1	3
4	3	1

$X=1$

$Y=i \quad (1 \le i < n)$

Number of times 1st comparison is true for a list of size n =
Number of times 1st comparison is true for a list of size (n-1)

$T(n)=T(n-1)$

Finding Min & Max

```
function MaxMin (L[0:n-1], Max, Min)
   Max ← L[0]
   Min ← L[0]
   for i←1 to n-1 do
      if L[i]>Max then
         Max ← L[i]
      else
         if L[i]<Min then
            Min ← L[i]
         endif
      endif
   endfor
end
```

- $A(n)$
  - $A(n) = 2n - 2 - \underbrace{\sum_{i=1}^{n} E[X|Y=i]P(Y=i)}$

$$E[X] = T(n) = \sum_{i=1}^{n} E[X|Y=i]P(Y=i)$$

$$= \left[\sum_{i=1}^{n-1} T(n-1)\frac{1}{n}\right] + (T(n-1)+1)\frac{1}{n}$$

$$= \frac{n-1}{n}T(n-1) + \frac{1}{n}T(n-1) + \frac{1}{n}$$

$$= T(n-1) + \frac{1}{n}$$

$$\Rightarrow T(n) = T(n-1) + \frac{1}{n} \quad , \quad T(1) = 0$$

$$\Rightarrow T(n) = H(n) - 1$$

$$\Rightarrow A(n) = 2n - 1 - H(n) \in \theta(n)$$

Strassen's Matrix Multiplication

- Multiply two $2X2$ matrices

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} * \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \qquad C = A * B$$

  - Classical algorithm

$c_{ij} = \sum_{k=1}^{2} a_{ik} b_{kj}$, $i, j = 1,2$ ⇒ 8 multiplications ⇒ $\theta(n^3)$ & 4 additions

  - A better algorithm

Strassen's algorithm (1969) ⇒ 7 multiplications ⇒ $\theta(n^{2.81})$ & … additions

Strassen's Matrix Multiplication

- Multiply two $2X2$ matrices

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} * \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \qquad C = A * B$$

  - Strassen's algorithm

Compute

$I = (a_{12} - a_{22}) * (b_{21} + b_{22})$
$II = (a_{11} + a_{22}) * (b_{11} + b_{22})$
$III = (a_{11} - a_{21}) * (b_{11} + b_{12})$
$IV = (a_{11} + a_{12}) * b_{22}$
$V = a_{11} * (b_{12} - b_{22})$
$VI = a_{22} * (b_{21} - b_{11})$
$VII = (a_{21} + a_{22}) * b_{11}$

Compute

$c_{11} = I + II - IV + VI$
$c_{12} = IV + V$
$c_{21} = VI + VII$
$c_{22} = II - III + V - VII$

Strassen's Matrix Multiplication

- Multiply two nXn matrices $(n = 2^k)$

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{nXn} = A_{nXn} * B_{nXn}$$

  - Divide *A* and *B* into four $2^{k-1}X2^{k-1}$ matrices
  - Consider *C* as composed of four $2^{k-1}X2^{k-1}$ matrices

    Each A_{ij} , B_{ij} , C_{ij} is a $2^{k-1}X2^{k-1}$ matrix
  - Use the computations given

    Replace scalar operations with matrix operations

Strassen's Matrix Multiplication

```
function MatrProd (A, B, n)
   if (n is not a power of 2) then
      append rows and columns of 0's to A and B
         until their sizes are the next power of 2
      change n accordingly
   endif
   if n=1 then
      return [A*B]1x1
   else
      partition each of A and B into four matrices
      I = MatrProd (A11-A22, B21+B22, n/2)
      ...
      VII = MatrProd (A21+A22, B11, n/2)
      C11 = I+II-IV+VI
      ...
      C22 = II-III+V-VII
      return [C11  C12]
             [C21  C22]nxn
   endif
end
```

7 recursive calls

18 calls to a matrix addition module

Strassen's Matrix Multiplication

- Analysis
  - Basic operation: multiplication

    $f(n)$: number of multiplications to multiply two nXn $(2^k X 2^k)$ matrices

  - $f(k) = 7 * f(k-1)$, $f(0) = 1$

$$\Rightarrow f(k) = 7^k$$
$$= 7^{\log_2 n}$$
$$= 7^{\log n / \log 2}$$
$$= n^{\log 7 / \log 2}$$
$$\cong n^{2.81}$$

$$\Rightarrow f(n) \in O(n^{2.81})$$

```
function MatrProd (A, B, n)
  if (n is not a power of 2) then
    append rows and columns of 0's to A and B
      until their sizes are the next power of 2
    change n accordingly
  endif
  if n=1 then
    return [A*B]1x1
  else
    partition each of A and B into four matrices
    I = MatrProd (A11-A22, B21+B22, n/2)
    ...
    VII = MatrProd (A21+A22, B11, n/2)
    C11 = I+II-IV+VI
    ...
    C22 = II-III+V-VII
    return [C11  C12]
           [C21  C22]nxn
  endif
end
```

Strassen's Matrix Multiplication

- Analysis
  - Operation: addition

    $g(n)$: number of additions to multiply two nXn $(2^k X 2^k)$ matrices

  - $g(k) = 7 * g(k-1) + 18 * \left(2^{k-1}\right)^2 \;, g(0) = 0$

$$\Rightarrow g(k) = 7 * g(k-1) + 18 * 4^{k-1}$$

$$g(k) = 7 * g(k-1) + \frac{9}{2} * 4^k$$

$$\frac{g(k)}{7^k} = \frac{g(k-1)}{7^{k-1}} + \frac{9}{2} * \left(\frac{4}{7}\right)^k$$

$y(k)$

$$\Rightarrow y(k) = y(k-1) + \frac{9}{2} * \left(\frac{4}{7}\right)^k \quad , \quad y(0) = 0$$

```
function MatrProd (A, B, n)
  if (n is not a power of 2) then
    append rows and columns of 0's to A and B
      until their sizes are the next power of 2
    change n accordingly
  endif
  if n=1 then
    return [A*B]1x1
  else
    partition each of A and B into four matrices
    I = MatrProd (A11-A22, B21+B22, n/2)
    ...
    VII = MatrProd (A21+A22, B11, n/2)
    C11 = I+II-IV+VI
    ...
    C22 = II-III+V-VII
    return [C11 C12; C21 C22]nxn
  endif
end
```

Standard matrix addition module
Adding two $2^{k-1}X2^{k-1}$ matrices

Strassen's Matrix Multiplication

- Analysis
  - Operation: addition

    $g(n)$: number of additions to multiply two nXn $(2^k X 2^k)$ matrices

  - $g(k) = 7 * g(k-1) + 18 * \left(2^{k-1}\right)^2 \ , g(0) = 0$

$$y(k) = y(k-1) + \frac{9}{2} * \left(\frac{4}{7}\right)^k$$

$$\Rightarrow y(k) = \frac{9}{2}\sum_{i=1}^{n}\left(\frac{4}{7}\right)^i \leq \frac{9}{2}\sum_{i=0}^{\infty}\left(\frac{4}{7}\right)^i = \frac{21}{2}$$

$$\Rightarrow g(k) = 7^k * y(k) \leq \frac{21}{2}7^k = \frac{21}{2}n^{2.81}$$

$$\Rightarrow g(n) \in O(n^{2.81})$$

```
function MatrProd (A, B, n)
  if (n is not a power of 2) then
    append rows and columns of 0's to A and B
      until their sizes are the next power of 2
    change n accordingly
  endif
  if n=1 then
    return [A*B]1x1
  else
    partition each of A and B into four matrices
    I = MatrProd (A11-A22, B21+B22, n/2)
    ...
    VII = MatrProd (A21+A22, B11, n/2)
    C11 = I+II-IV+VI
    ...
    C22 = II-III+V-VII
    return [C11  C12]
           [C21  C22]nxn
  endif
end
```

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x} \ , \ |x| < 1$$

Strassen's Matrix Multiplication

- Theoretical importance
  - First that decreased the classical $O(n^3)$ complexity
  - State-of-the-art

    $O(n^{2.37})$ complexity

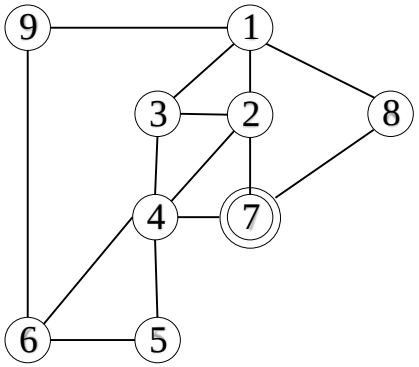
    Le Gall, F., "Powers of tensors and fast matrix multiplication", Proc. of the 39th Int. Symposium on Symbolic and Algebraic Computation (ISSAC), p.296-303, 2014

Graph Search/Traversal

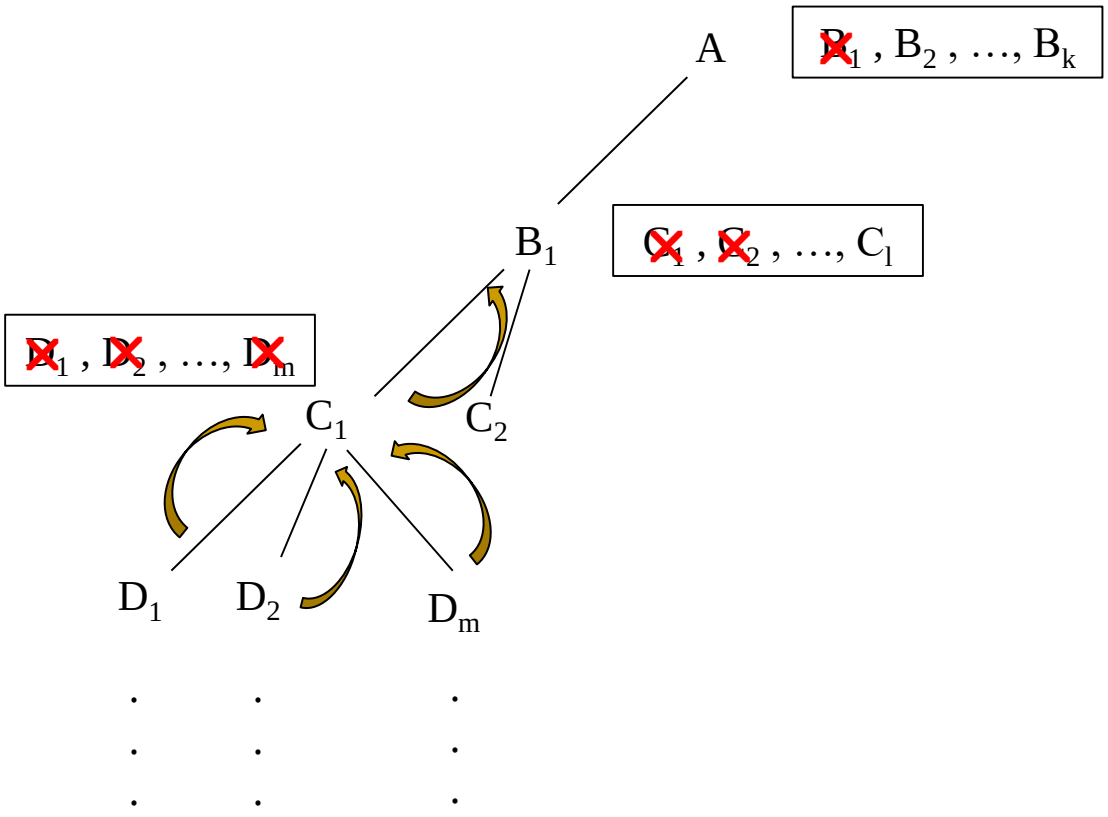
- Operations on trees/graphs
  - Search: Accessing (visiting) some nodes in the tree/graph
  - Traverse: Accessing (visiting) all the nodes in the tree/graph
- Techniques
  - Depth-first search (DFS)
  - Breadth-first search (BFS)

Graph Search/Traversal

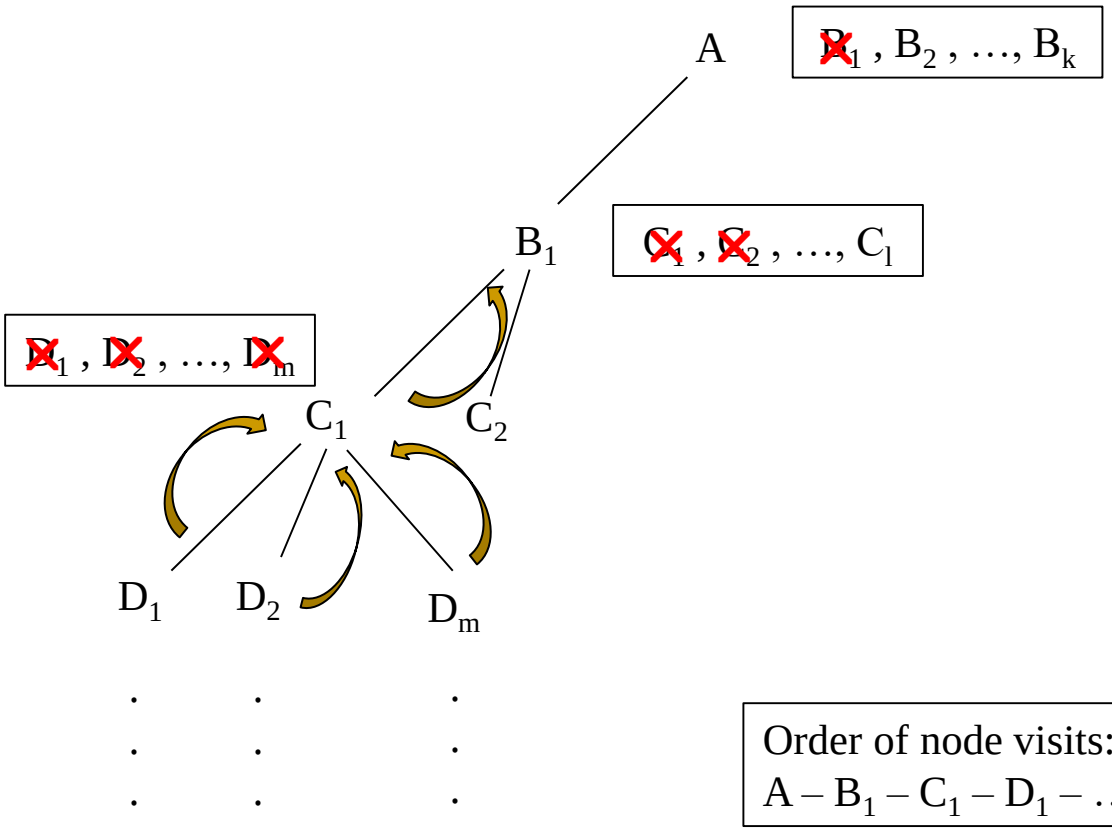
- Example



DFS



DFS

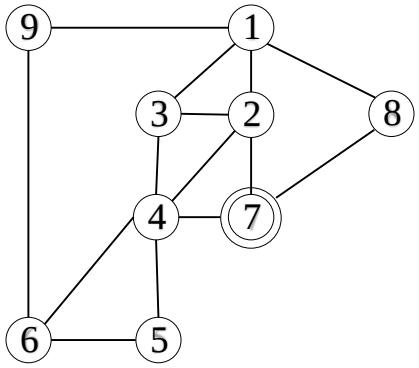


Order of node visits:
A – B_1 – C_1 – D_1 – … – D_2 – … – D_m – … – C_2 – …

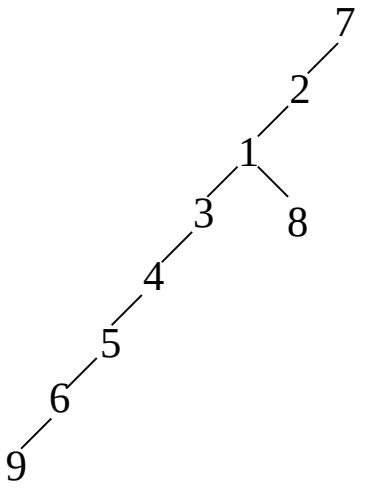
DFS

Visit	Unvisited neighbors	Backtrack
7	2, 4, 8	
2	1, 3, 4	
1	3, 8, 9	
3	4	
4	5, 6	
5	6	
6	9	
9	---	to 6
6 (returned)	---	to 5
5 (returned)	---	to 4
4 (returned)	---	to 3
3 (returned)	---	to 1
1 (returned)	8	
8	---	to 1
1 (returned)	---	to 2
2 (returned)	---	to 7
7 (returned)	---	Terminate

Order of node visits:
7 – 2 – 1 – 3 – 4 – 5 – 6 – 9 – 8



DFS-tree



DFS

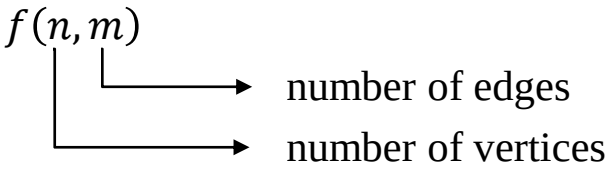
```
function DFS (G, v)
   call Visit (v)
   Mark [v] ← 1
   u ← v
   call Next (u, w, found)
   while (found) or (not Empty (Stack))
      if found then
         call Push (Stack, u)
         call Visit (w)
         Mark [w] ← 1
         u ← w
      else
         call Pop (Stack, u)
      endif
      call Next (u, w, found)
   endwhile
end
```

Implementation:

- Stack
- Recursion

DFS

- Analysis
  - Input parameters



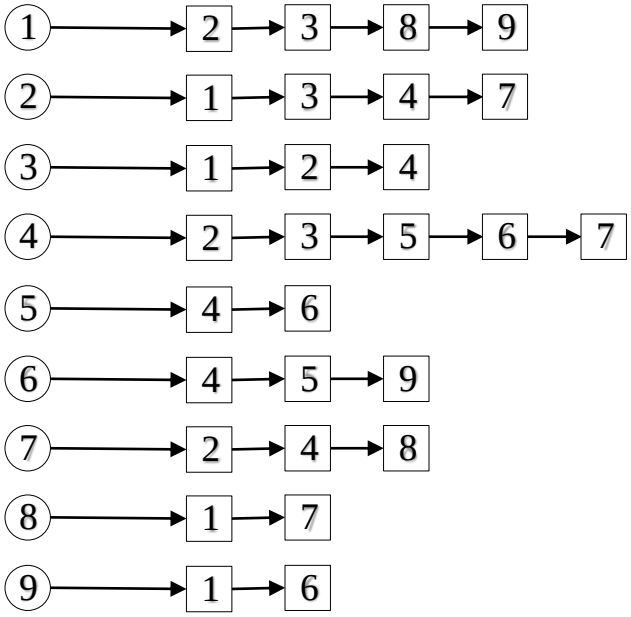
  - Basic operation:

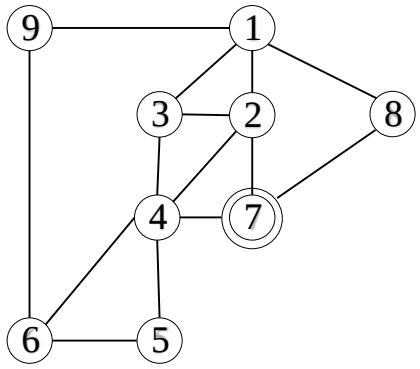
Visiting a node + Accessing an edge

```
function DFS (G, v)
   call Visit (v)
   Mark [v] ← 1
   u ← v
   call Next (u, w, found)
   while (found) or (not Empty (Stack))
      if found then
         call Push (Stack, u)
         call Visit (w)
         Mark [w] ← 1
         u ← w
      else
         call Pop (Stack, u)
      endif
      call Next (u, w, found)
   endwhile
end
```

DFS

Adjacency List

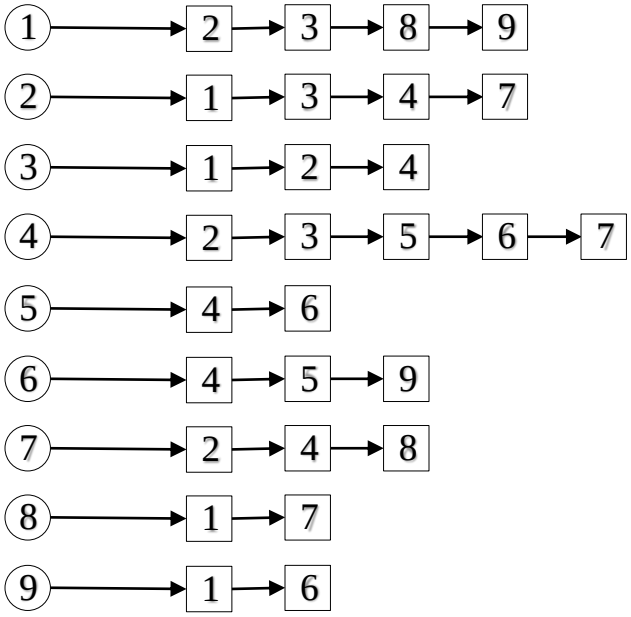




DFS

Node visits
&
Edge accesses

Adjacency List

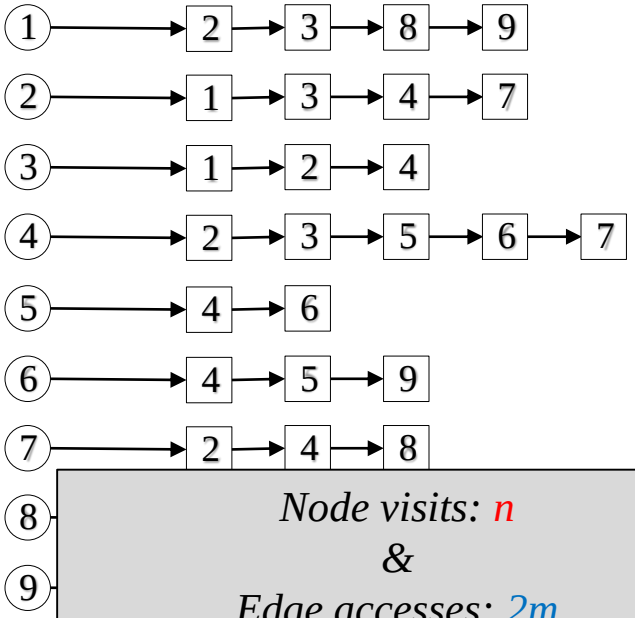


Node visits	Edge accesses
7	$E_{7\to2}$ (2 not visited, goto 2, delete $E_{7\to2}$)
2	$E_{2\to1}$ (1 not visited, goto 1, delete $E_{2\to1}$)
1	$E_{1\to2}$ (2 visited, delete $E_{1\to2}$) $E_{1\to3}$ (3 not visited, goto 3, delete $E_{1\to3}$)
3	$E_{3\to1}$ (1 visited, delete $E_{3\to1}$) $E_{3\to2}$ (2 visited, delete $E_{3\to2}$) $E_{3\to4}$ (4 not visited, goto 4, delete $E_{3\to4}$)
4	$E_{4\to2}$ (2 visited, delete $E_{4\to2}$) $E_{4\to3}$ (3 visited, delete $E_{4\to3}$) $E_{4\to5}$ (5 not visited, goto 5, delete $E_{4\to5}$)
5	$E_{5\to4}$ (4 visited, delete $E_{5\to4}$) $E_{5\to6}$ (6 not visited, goto 6, delete $E_{5\to6}$)
6	$E_{6\to4}$ (4 visited, delete $E_{6\to4}$) $E_{6\to5}$ (5 visited, delete $E_{6\to5}$) $E_{6\to9}$ (9 not visited, goto 9, delete $E_{6\to9}$)
9	$E_{9\to1}$ (1 visited, delete $E_{9\to1}$) $E_{9\to6}$ (6 visited, delete $E_{9\to6}$) Backtrack to the node popped (6) Backtrack to the node popped (5) Backtrack to the node popped (4) $E_{4\to6}$ (6 visited, delete $E_{4\to6}$) $E_{4\to7}$ (7 visited, delete $E_{4\to7}$) Backtrack to the node popped (3) Backtrack to the node popped (1) $E_{1\to8}$ (8 not visited, goto 8, delete $E_{1\to8}$)
8	$E_{8\to1}$ (1 visited, delete $E_{8\to1}$) $E_{8\to7}$ (7 visited, delete $E_{8\to7}$) Backtrack to the node popped (1) $E_{1\to9}$ (9 visited, delete $E_{1\to9}$) Backtrack to the node popped (2) $E_{2\to3}$ (3 visited, delete $E_{2\to3}$) $E_{2\to4}$ (4 visited, delete $E_{2\to4}$) $E_{2\to7}$ (7 visited, delete $E_{2\to7}$) Backtrack to the node popped (7) $E_{7\to4}$ (4 visited, delete $E_{7\to4}$) $E_{7\to8}$ (8 visited, delete $E_{7\to8}$)

DFS

Node visits
&
Edge accesses

Adjacency List

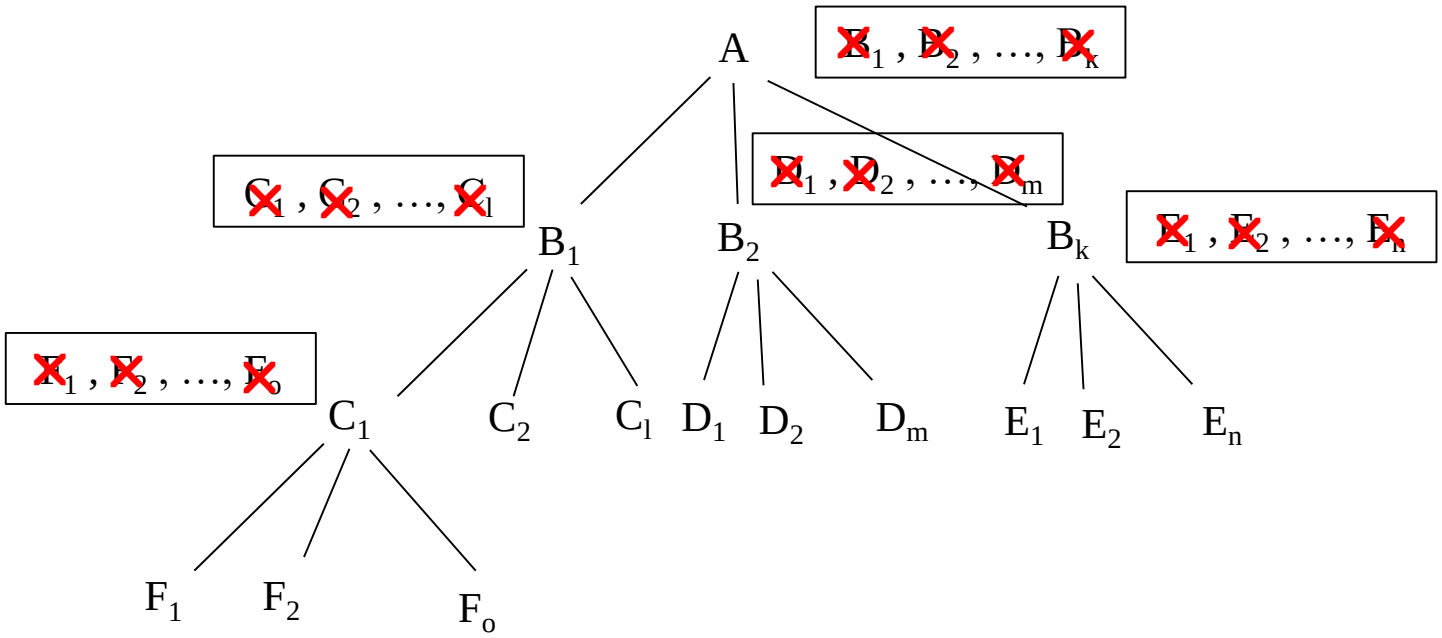


Node visits: n
&
Edge accesses: $2m$

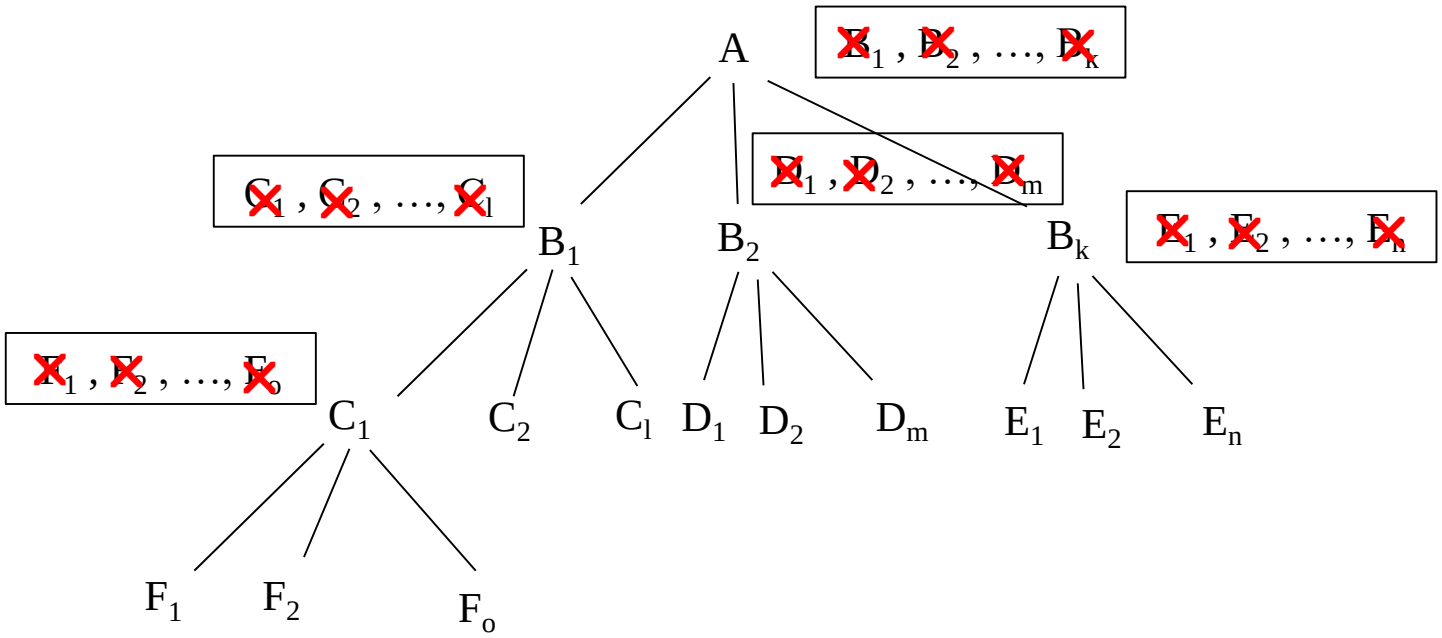
$$\Rightarrow W(n,m) = n + 2m \in \theta(n+m)$$

Node visits	Edge accesses
7	$E_{7\to2}$ (2 not visited, goto 2, delete $E_{7\to2}$)
2	$E_{2\to1}$ (1 not visited, goto 1, delete $E_{2\to1}$)
1	$E_{1\to2}$ (2 visited, delete $E_{1\to2}$) $E_{1\to3}$ (3 not visited, goto 3, delete $E_{1\to3}$)
3	$E_{3\to1}$ (1 visited, delete $E_{3\to1}$) $E_{3\to2}$ (2 visited, delete $E_{3\to2}$) $E_{3\to4}$ (4 not visited, goto 4, delete $E_{3\to4}$)
4	$E_{4\to2}$ (2 visited, delete $E_{4\to2}$) $E_{4\to3}$ (3 visited, delete $E_{4\to3}$) $E_{4\to5}$ (5 not visited, goto 5, delete $E_{4\to5}$)
5	$E_{5\to4}$ (4 visited, delete $E_{5\to4}$) $E_{5\to6}$ (6 not visited, goto 6, delete $E_{5\to6}$)
6	$E_{6\to4}$ (4 visited, delete $E_{6\to4}$) $E_{6\to5}$ (5 visited, delete $E_{6\to5}$) $E_{6\to9}$ (9 not visited, goto 9, delete $E_{6\to9}$)
9	$E_{9\to1}$ (1 visited, delete $E_{9\to1}$) $E_{9\to6}$ (6 visited, delete $E_{9\to6}$) Backtrack to the node popped (6) Backtrack to the node popped (5) Backtrack to the node popped (4) $E_{4\to6}$ (6 visited, delete $E_{4\to6}$) $E_{4\to7}$ (7 visited, delete $E_{4\to7}$) Backtrack to the node popped (3) Backtrack to the node popped (1) $E_{1\to8}$ (8 not visited, goto 8, delete $E_{1\to8}$)
8	$E_{8\to1}$ (1 visited, delete $E_{8\to1}$) $E_{8\to7}$ (7 visited, delete $E_{8\to7}$) Backtrack to the node popped (1) $E_{1\to9}$ (9 visited, delete $E_{1\to9}$) Backtrack to the node popped (2) $E_{2\to3}$ (3 visited, delete $E_{2\to3}$) $E_{2\to4}$ (4 visited, delete $E_{2\to4}$) $E_{2\to7}$ (7 visited, delete $E_{2\to7}$) Backtrack to the node popped (7) $E_{7\to4}$ (4 visited, delete $E_{7\to4}$) $E_{7\to8}$ (8 visited, delete $E_{7\to8}$)

BFS


A
B1 , B2 , …, Bk
C1 , C2 , …, Cl
D1 , D2 , …, Dm
E1 , E2 , …, En
F1 , F2 , …, Fo
B1
B2
Bk
C1
C2
Cl
D1
D2
Dm
E1
E2
En
F1
F2
Fo

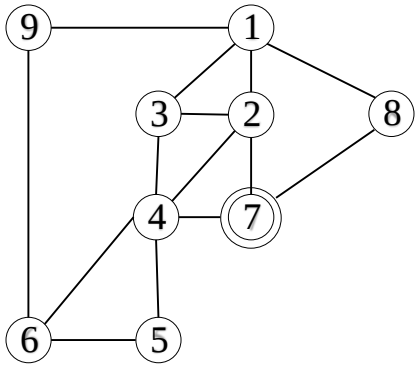
BFS



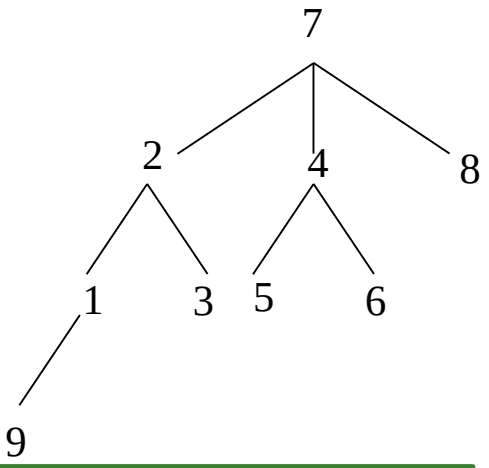
Order of node visits:
$A - B_1 - \ldots - B_k - C_1 - \ldots - C_l - D_1 - \ldots - D_m - E_1 - \ldots - E_n - F_1 - \ldots - F_o - \ldots$

BFS

Visit	Unvisited neighbors	Enqueue
7	2, 4, 8	
2		2
4		4
8		8
2 (dequeue)	1, 3	
1		1
3		3
4 (dequeue)	5, 6	
5		5
6		6
8 (dequeue)	---	
1 (dequeue)	9	
9		9
3 (dequeue)	---	
5 (dequeue)	---	
6 (dequeue)	---	
9 (dequeue)	---	Terminate



BFS-tree



Order of node visits:
7 – 2 – 4 – 8 – 1 – 3 – 5 – 6 – 9

Shortest path tree
(rooted at 7)

BFS

```
function BFS (G, v)
   call Visit (v)
   Mark [v] ← 1
   call Enqueue (Queue, v)
   while (not Empty (Queue))
      Dequeue (Queue, u)
      for each vertex w adjacent to u do
         if Mark [w] = 0 then
            call Enqueue (Queue, w)
            Mark [w] ← 1
            call Visit (w)
         endif
      endfor
   endwhile
end
```

Implementation:

- Queue

BFS

- Analysis
  - Input parameters

$f(n, m)$

- n → number of edges
- m → number of vertices

  - Basic operation:

Visiting a node + Accessing an edge

Node visits: n
&
Edge accesses: $2m$

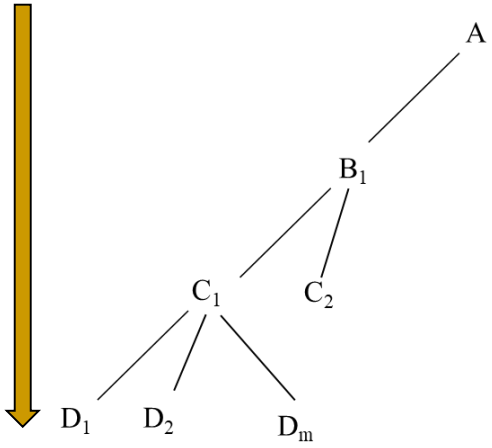
$\Rightarrow W(n, m) = n + 2m \in \theta(n + m)$

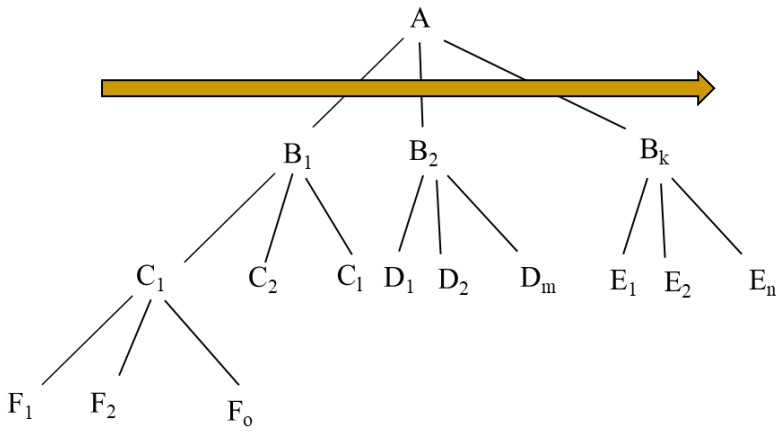
```
function BFS (G, v)
   call Visit (v)
   Mark [v] <-- 1
   call Enqueue (Queue, v)
   while (not Empty (Queue))
      Dequeue (Queue, u)
      for each vertex w adjacent to u do
         if Mark [w] = 0 then
            call Enqueue (Queue, w)
            Mark [w] <-- 1
            call Visit (w)
         endif
      endfor
   endwhile
end
```

SPP lower bound $\Omega(n + m)$

$\Rightarrow$ Optimal SPP algorithm

DFS & BFS





Applications:
- Topological sorting
- Path finding
- Cycle detection in a graph
- Test if a graph is bipartite
- …

Applications:
- Shortest path
- Minimum spanning tree
- Garbage collection
- Network applications
- …

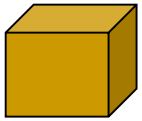
Knapsack Problem

- Initial:

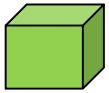
Knapsack
Capacity: C



Object o_0
Weight w_0
Value v_0

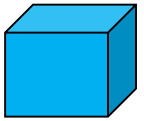


Object o_1
Weight w_1
Value v_1



…

Object o_{n-1}
Weight w_{n-1}
Value v_{n-1}



- Goal:



Fractional knapsack problem:

Total weight of objects in knapsack $\leq C$

Total value of objects in knapsack $\rightarrow$ max

Knapsack Problem

- Initial:

Knapsack
Capacity: C

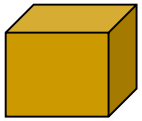


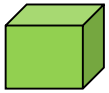
Object o_0
Weight w_0
Value v_0

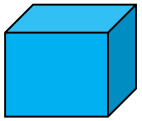
Object o_1
Weight w_1
Value v_1

…

Object o_{n-1}
Weight w_{n-1}
Value v_{n-1}







- Goal:



Formal definition:

$$maximize \ \sum_{i=0}^{n-1} f_i \, v_i$$

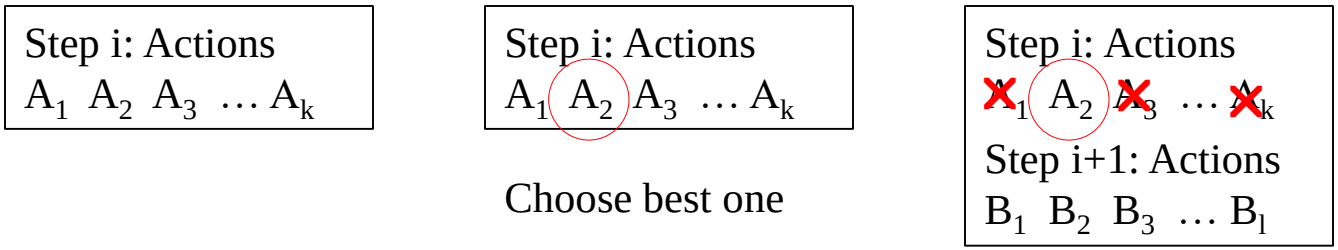
$$subject \ to \ \sum_{i=0}^{n-1} f_i \, w_i \leq C$$

$$0 \leq f_i \leq 1 \quad , i = 0, \ldots, n-1$$

Knapsack Problem

- Optimization problem

- Can be solved using different techniques
  - Greedy
  - Dynamic programming
  - …



Knapsack Problem

```
function Knapsack (V[0:n-1], W[0:n-1], C, F[0:n-1])
    Sort V[0:n-1] and W[0:n-1] in decreasing order of
        V[i]/W[i] values
    for i←0 to n-1 do
        F[i] ← 0
    endfor
    RemainCap ← C
    i ← 0
    if W[0] ≤ C then
        Fits ← true
    else
        Fits ← false
    endif
    while Fits and i ≤ n-1 do
        F[i] ← 1
        RemainCap ← RemainCap – W[i]
        i ← i+1
        if W[i] ≤ RemainCap then
            Fits ← true
        else
            Fits ← false
        endif
    endwhile
    if i ≤ n-1 then
        F[i] ← RemainCap / W[i]
    endif
end
```

How to design a greedy knapsack algorithm?

- Sort wrt $\frac{v}{w}$ ratios
- Place according to ratios

Greedy?

Knapsack Problem

- Example
  - $C = 10$

	o_0	o_1	o_2	o_3	o_4	o_5	o_6	o_7
W	8	2	4	1	2	6	10	50
V	32	32	25	10	9	18	55	100




```
function Knapsack (V[0:n-1], W[0:n-1], C, F[0:n-1])
   Sort V[0:n-1] and W[0:n-1] in decreasing order of
      V[i]/W[i] values
   for i←0 to n-1 do
      F[i] ← 0
   endfor
   RemainCap ← C
   i ← 0
   if W[0] ≤ C then
      Fits ← true
   else
      Fits ← false
   endif
   while Fits and i ≤ n-1 do
      F[i] ← 1
      RemainCap ← RemainCap – W[i]
      i ← i+1
      if W[i] ≤ RemainCap then
         Fits ← true
      else
         Fits ← false
      endif
   endwhile
   if i ≤ n-1 then
      F[i] ← RemainCap / W[i]
   endif
end
```

RemainCap : 0

Total value:
1*32+1*10+1*25+0.3*55=83.5

Knapsack Problem

- Optimal solution

- Complexity

  - Basic operation

    Comparison in sorting

    &

    Assignment in while loop

  - $W(n)$

    While loop $\rightarrow n$

    $\Rightarrow \theta(n \log n)$ & $\theta(n)$

    $\Rightarrow W(n) \in \theta(n \log n)$

```
function Knapsack (V[0:n-1], W[0:n-1], C, F[0:n-1])
   Sort V[0:n-1] and W[0:n-1] in decreasing order of
      V[i]/W[i] values
   for i←0 to n-1 do
      F[i] ← 0
   endfor
   RemainCap ← C
   i ← 0
   if W[0] ≤ C then
      Fits ← true
   else
      Fits ← false
   endif
   while Fits and i ≤ n-1 do
      F[i] ← 1
      RemainCap ← RemainCap – W[i]
      i ← i+1
      if W[i] ≤ RemainCap then
         Fits ← true
      else
         Fits ← false
      endif
   endwhile
   if i ≤ n-1 then
      F[i] ← RemainCap / W[i]
   endif
end
```

Knapsack Problem

- 0/1 knapsack problem

Formal definition:

$$maximize \sum_{i=0}^{n-1} f_i \, v_i$$

$$subject\ to \sum_{i=0}^{n-1} f_i \, w_i \leq C$$

$$0 \leq f_i \leq 1 \quad , i = 0, \ldots, n-1$$



Formal definition:

$$maximize \sum_{i=0}^{n-1} f_i \, v_i$$

$$subject\ to \sum_{i=0}^{n-1} f_i \, w_i \leq C$$

$$f_i \in \{0,1\} \quad , i = 0, \ldots, n-1$$

Knapsack Problem

- 0/1 knapsack problem
  - May miss the optimal solution

Example: $C = 6$

	o_0	o_1	o_2
W	4	3	3
V	5	3	3

```
function Knapsack (V[0:n-1], W[0:n-1], C, F[0:n-1])
    Sort V[0:n-1] and W[0:n-1] in decreasing order of
        V[i]/W[i] values
    for i←0 to n-1 do
        F[i] ← 0
    endfor
    RemainCap ← C
    i ← 0
    if W[0] ≤ C then
        Fits ← true
    else
        Fits ← false
    endif
    while Fits and i ≤ n-1 do
        F[i] ← 1
        RemainCap ← RemainCap – W[i]
        i ← i+1
        if W[i] ≤ RemainCap then
            Fits ← true
        else
            Fits ← false
        endif
    endwhile
```



RemainCap : 2

Total value:
1*5=5



RemainCap : 0

Total value:
1*3+1*3=6

Limitations of Asymptotic Analysis

- Limitation 1
  - Asymptotic analysis ignores everything other than the basic operation
  - Example: Comparison-based sorting

    Algorithm $A \rightarrow \theta(n^2)$ comparisons & $\theta(n^2)$ copying operations

    Algorithm $B \rightarrow \theta(n^2)$ comparisons & $\theta(n)$ copying operations

    Algorithm $C \rightarrow \theta(n^2)$ comparisons & $\theta(n)$ pointer assignments

~~Best one?~~
~~All the same~~

Best one?
Algorithm *C*

Limitations of Asymptotic Analysis

- Limitation 2
  - Asymptotic analysis hides low-order terms and constant factors
  - Example:

Algorithm $A \rightarrow f(n) = 1000000n + 1000 \log n \Rightarrow \theta(n)$ linear

Algorithm $B \rightarrow g(n) = \frac{n^2}{10} \Rightarrow \theta(n^2)$ quadratic

~~Asymptotic analysis: Algorithm *A*~~

In practice: Algorithm *B*